

Module 11: AI Architecture - RAG, Chains and Agents

Charles S. Givre CISSP

Outline

- Part 1: Introduction and Theory: How Generative Models Work
- Part 2: Prompt Engineering: How to get the most out of Generative Models
- **Part 3: Architecture: How to build AI driven applications**
- Part 4: Red Teaming: How to Attack & Defend AI Applications

Module Outline

- AI Components
- Retrieval Augmented Generation (RAG)
- Tools & Skills
- LangChain
- Building AI Agents

Setup

- This section has a series of non-code based labs.
- The labs are available at https://ai.gtkcyber.com.
- Please take 5 minutes and create an account there.

Retrieval Augmented Generation (RAG)

Big Problem with LLMs: Building an LLM to work with proprietary data.

Problems with LLMs

- Training takes a long time and is expensive.
- It is a practical impossibility to keep models up to date with the latest information, especially for areas with fast moving data.
- If you are using public models, you cannot use proprietary data in queries.

1. Train a custom model

2. Fine Tune a Foundational Model

Fine Tuning

- Fine tuning allows you to tune an existing model for your specific use cases.
- Drawbacks are that it can be time consuming and expensive.
- You will need a decent amount of data for fine tuning.

RAG to the R



What is RAG?

- RAG stands for Retrieval Augmented Generation and bridges the gap between static models and dynamic systems.
- RAG allows an organization to use AI to generate responses based on either proprietary or current information without having to retrain a model or fine tune it.

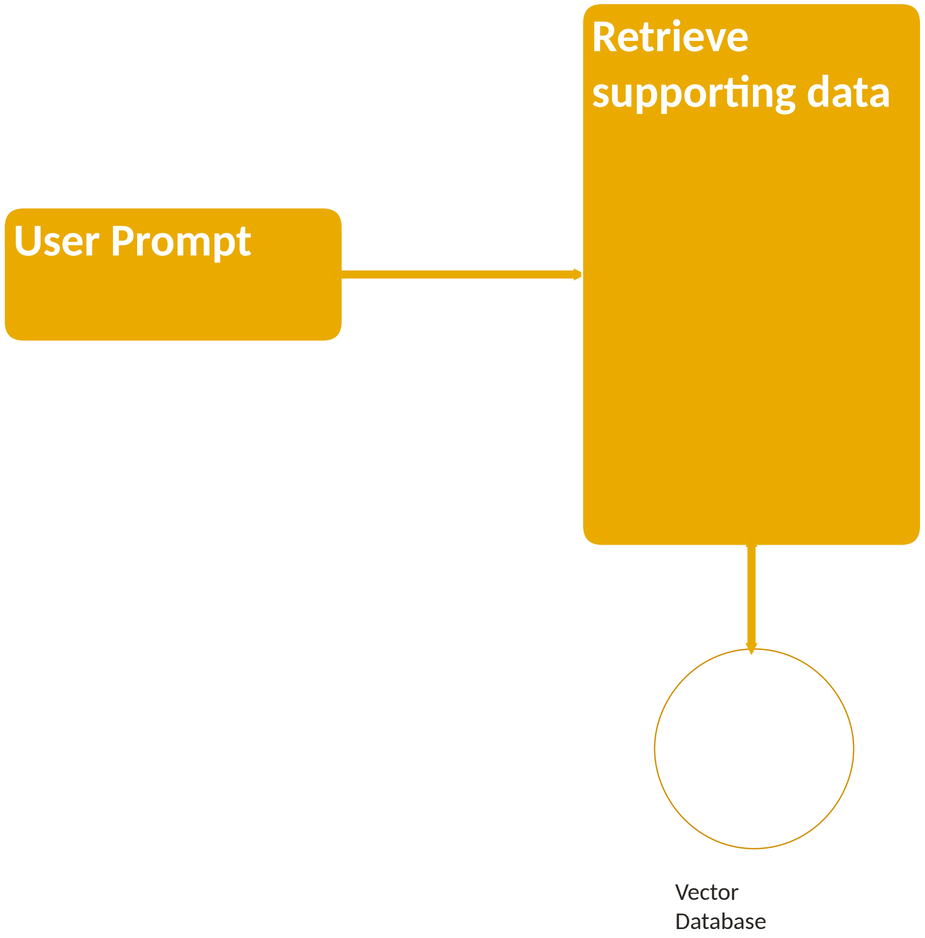
How RAG works

User Prompt

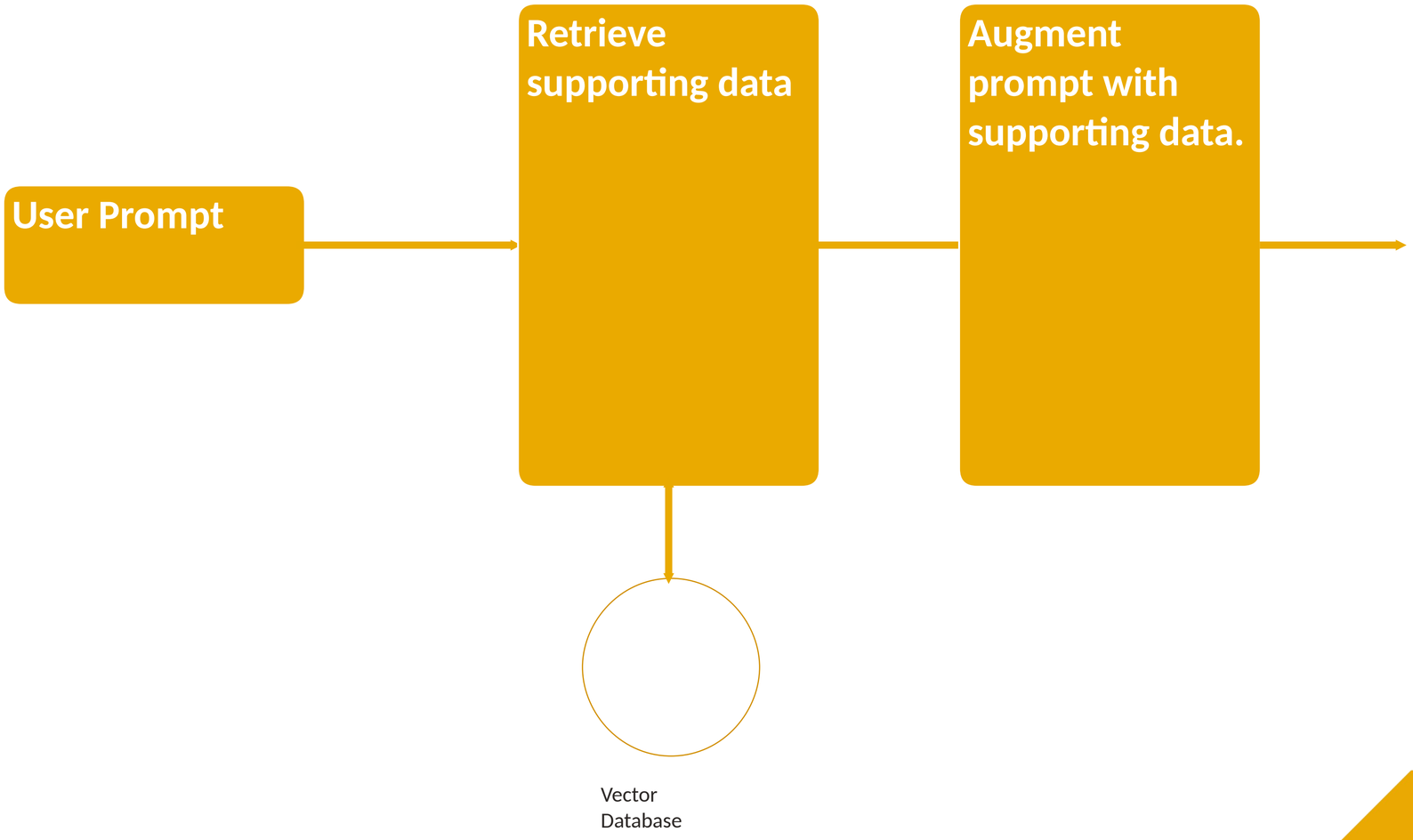


Vector
Database

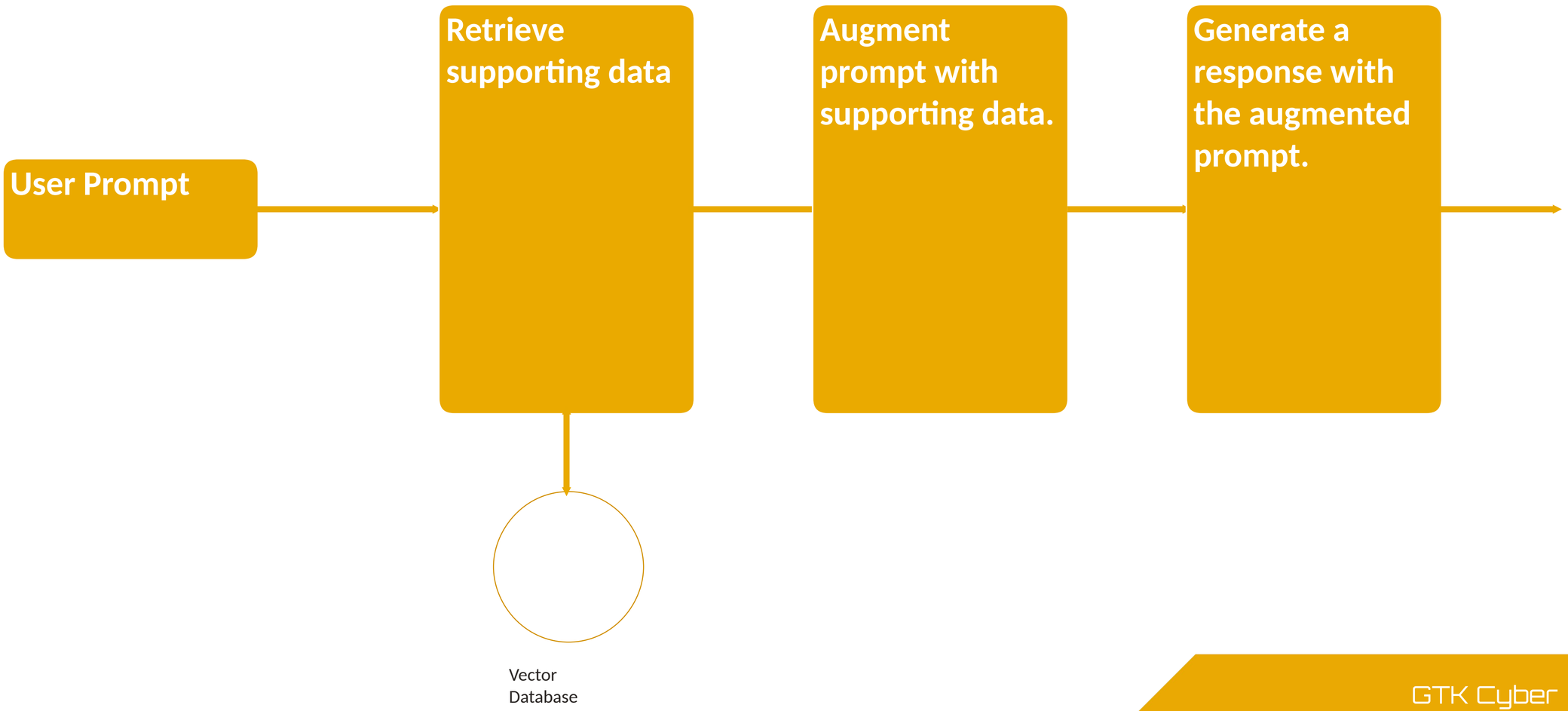
How RAG works



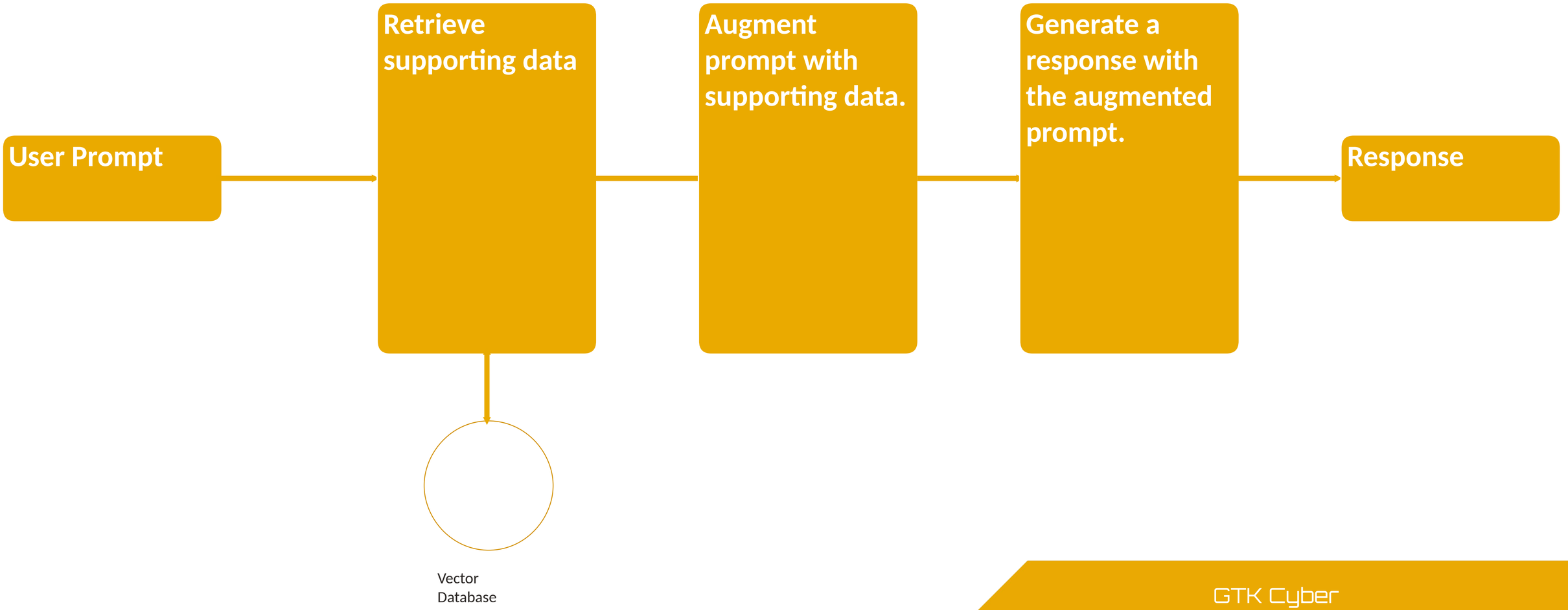
How RAG works



How RAG works



How RAG works



Components of a RAG workflow

- **Embedding Model**: Converts the prompt into embeddings
- **Vector Database**: A live repository which contains live, up to date information for the model to retrieve. Vector databases are specialty databases for this purpose and quickly match your query vector to the most similar vectors in the database.
- **LLM**: A generative large language model generates the actual response.

Be consistent with your embeddings

RAG in Cyber

- RAG is a great tool for any AI driven applications which involve proprietary or changing information.
- Compliance is an excellent use case.
- Explaining MITRE ATT&CK techniques and comparing them to log info.

Vector Databases

- Vector databases are purpose built databases for retrieval of semantically similar vectors. They implement nearest neighbor algorithms.
- Popular examples now are ChromaDB and Pinecone.
- TurboVec is an interesting new project which is significantly faster than other vector DBs.
- Many databases can be used as a Vector store including MySQL, MongoDB, Oracle, Postgres and others.

Vector Databases

- We will be using Chroma for our examples and labs.
- Chroma is an OSS vector database which has become quite popular for AI use cases.
- Docs available here: https://docs.trychroma.com/docs/overview/introduction.
- For production use cases, you will need to set up Chroma in the traditional manner, but for our labs, that is not necessary.

Chroma Basic Usage

Chroma organizes data into key/values and is capable of rapid retrieval and efficient storage.

```
import chroma
# Create the client
chroma_client = chromadb.Client()

# Create a document collection if necessary
collection = chroma_client.create_collection(name="collection1")
```

https://docs.trychroma.com/docs/overview/getting-started

Chroma Basic Usage

```
# You can provide an embedding function or have chroma do it.

collection.add(
    ids=["id1", "id2"],
    documents=[
        "This is a document about pineapple",
        "This is a document about oranges"
    ]
)
```

https://docs.trychroma.com/docs/overview/getting-started

Chroma Basic Usage

```
# Here's where the magic happens...
results = collection.query(

          # Chroma will embed this for you
   query_texts=["I really like citrus"],

             n_results=1 # how many results to return
)

{'ids': [['id2']],
  'embeddings': None,
  'documents': [['This is a document about oranges']],
  'uris': None,
  'included': ['metadatas', 'documents', 'distances'],
  'data': None,
  'metadatas': [[None]],
  'distances': [[1.0216939449310303]]
}
```

https://docs.trychroma.com/docs/overview/getting-started

Building The Prompt for RAG

- Once you have relevant documents from the vector database, you still have to craft a prompt that includes the contextual documents from the vector database.
- The prompt might look something like this:

```
template = f"""Use the following pieces of context to answer the question at the end. If you don't know the answer, just say that you don't know. Use three sentences maximum. Keep the answer as concise as possible.

{context}

Question: {question}

Helpful Answer:"""
```

Remember to keep your prompts short and to the point.

RAG Tips

- RAG queries can have the advantage of providing sourcing information. Seeing sourcing is a good indicator that an AI application is using a RAG pipeline.
- Inputs to RAG pipelines can be poisoned. (More on this later)
- You will have to develop a strategy for chunking documents if you are using large documents.
- RAG can be expensive because it can blow up the input prompt size.

Chunking

- If you have large documents, you will have to divide them up into chunks.
- A good chunking strategy can mean the difference between success and failure for a RAG ap|



Implementing a RAG Pipeline

Lab

Please complete RAG Lab: https://ai.gtkcyber.com/rag-lab

For experimentation purposes in the ai_examples folder there is a tesla_aurora.txt file you can use for the RAG lab.

Privacy Still Matters with RAG

- RAG pipelines put the similar documents in the prompt.
- If you are using a public model, those documents are being sent to the model owner in the prompt.



Challenges Building AI Applications

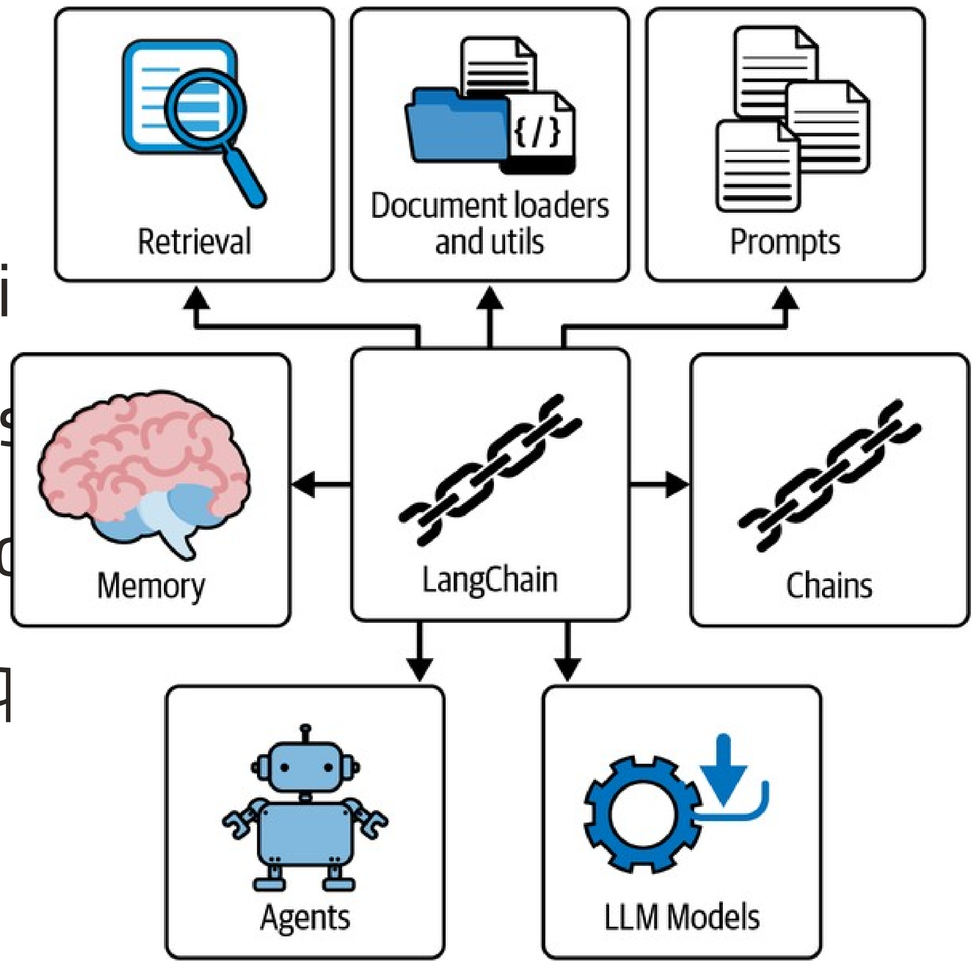
- Models have different SDKs and constructing pipelines.
- As you build out more complex applications, there are an increasing number of steps in that pipeline.
- To build agentic applications, your LLM needs to be able to interact with data and other downstream steps.
- How do we do this?

Introducing LangChain

- LangChain is a framework designed to facilitate building AI applications by abstracting the common components, and having a lot of different implementations of these components.
- This is a VERY fast moving area and there are some other new frameworks such as n8n which are also designed to facilitate building AI driven applications.
- There are commercial products such as Make.com, String.com, Zapier and others.

LangChain Modules

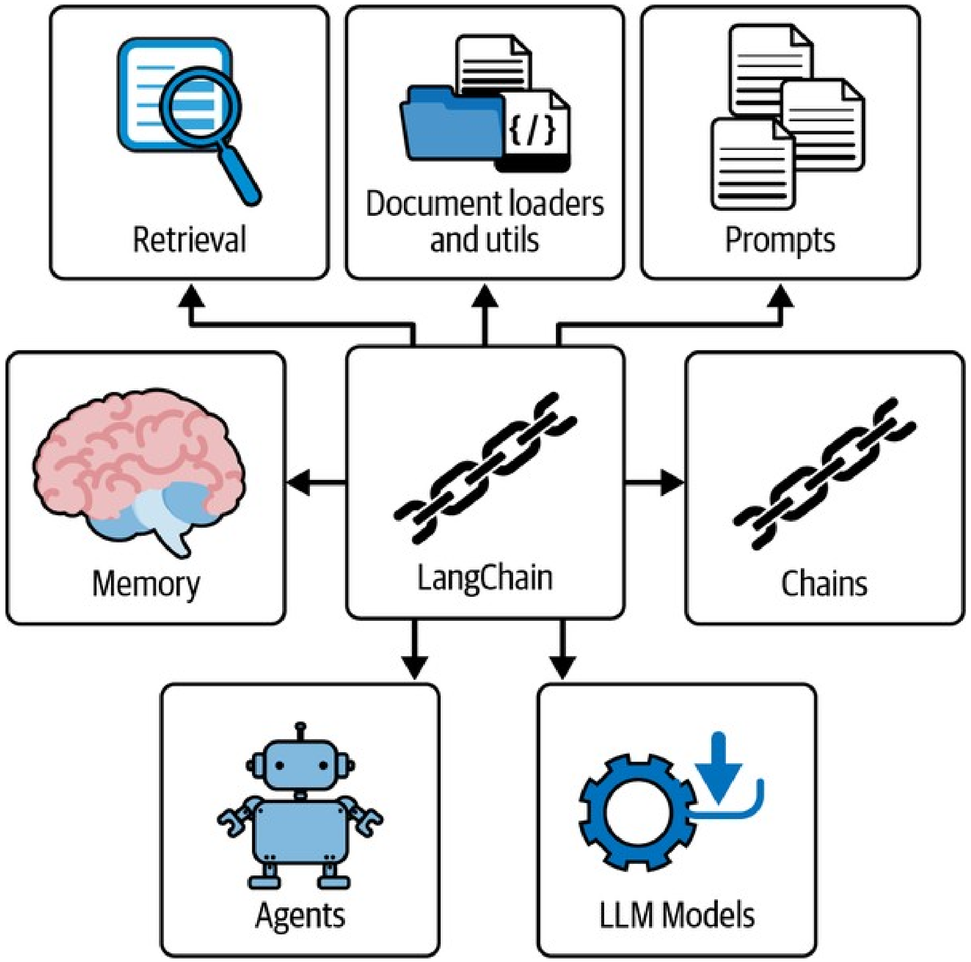
- LangChain features 6 top-level abstracti
  - **Model I/O**: Facilitates inputs/outputs
  - **Retrieval**: Retrieving relevant text fr
  - **Chains**: Enable the contraction of seq



Im
ag
e
fro

LangChain Modules

- **Agents:** Allow chains to make decisions on which tools to use based on high level instructions.
- **Memory**: Saves the state of the application between LLM interactions.
- **Callbacks:** Used to run additional code on an event happening
- **Models:** Abstracts a LLM



Im
ag
e
fro

LangChain Messages

- There are three message abstractions in LangChain:
  - SystemMessage: This is the message which tells the model how it should behave
  - HumanMessage: This is information coming from the user that the AI responds to
  - AIMessage: The responses from the AI.

LangChain Example 1

Note: All the LangChain examples can be found in the `ai_examples` folder in the class repo.

```
from langchain_openai.chat_models import ChatOpenAI
from langchain.schema import HumanMessage, SystemMessage

chat = ChatOpenAI(temperature=0.75,
                  model="gpt-3.5-turbo",
                  api_key=os.getenv("OPENAI_KEY"))

messages = [
   SystemMessage(content="You are a helpful assistant that generates ASCII art.  You must only respond with a witty greeting and a cheerful ASCII art based on the user's input."),
   HumanMessage(content="Hello, how are you?  It sure is beautiful outside!"),
]

response = chat.invoke(messages)
print(response)
```

LangChain Example 1

```
'usage_metadata': {'input_token_details': {'audio': 0,
'cache_read': 0},
                   'input_tokens': 55,
                   'output_token_details': {'audio': 0,
'reasoning': 0},
                   'output_tokens': 159,
                   'total_tokens': 214}}
```



LangChain and Streaming

- Generative models generate one token at a a time. To reduce latency, you can stream the results from the model instead of waiting for the entire completion.
- LangChain supports this, but it adds some complexity to your code and is beyond the scope of this class.
- You can also send batches (in parallel) to LangChain if you want to generate multiple responses.

LangChain Prompt Templates

- LangChain has a prompt template object which are templates that accept parameters and construct a prompt for the LLM.
- LangChain prompt templates allow you to easily validate user inputs, combine prompts together and easily create few-shot examples from data.

```
system_template = "Your name is {ai_name}"
system_prompt = SystemMessageTemplate.from_template(template)
chat_prompt = ChatPromptTemplate.from_messages([system_prompt])
chain = chat_prompt | model
result = chain.invoke({ai_name: "Jill"})
# Take a look at risky_sql_queries.py
```

LangChain Expression Language (LCEL)

- LCEL is a declarative language for building complex runnable AI applications
- It's the glue that puts all the pieces together.
- LCEL offers benefits such as optimized parallel execution, asynchronous support, and simplified streaming.
- LCEL runnables are logged automatically and feature a standard API for ease of development.

https://python.langchain.com/docs/concepts/lcel/

Single LLM Calls do not need LCEL

LCEL Runnable Sequences

- LCEL has two types of runnable objects: RunnableSequence and RunnableParallel
- Both can be run synchronously or asynchronously.
- You can call a chain using the `invoke()` method.
- LCEL overloaded the | operator so any runnable objects can be chained together with the | as shown below:

```
chain = step1 | step2 | step3
```

LangChain Output Parsers

- LangChain has a collection of Output Parsers which can extract structured data from LLM responses. Currently available parsers are:
  - List Parser
  - Datetime Parser
  - Auto-fixing Parser
  - Pydantic Parser (Predefined JSON Objects)
  - Retry Parser
  - Structured Output Parser
  - XML Parser
  - More...
- LangChain Parsers can generate output instructions for the LLM!

Tangent: Pydantic

- Pydantic is a data validation library which you can use to define schemata for complex JSON objects.
- The easy way to create Pydantic objects are tools like https://jsontopydantic.com/

Using an Output Formatter

```
from langchain_openai.chat_models import ChatOpenAI
from langchain_core.prompts import SystemMessagePromptTemplate, ChatPromptTemplate
from langchain_core.output_parsers import PydanticOutputParser
from pydantic import BaseModel

class SQL_Query(BaseModel):
    query: str
    is_malicious: bool
    table_count: int

json_output_parser = PydanticOutputParser(pydantic_object=SQL_Query)
json_output_parser.get_format_instructions()
```

Using an Output Formatter

```
The output should be formatted as a JSON instance that conforms to the JSON schema below.


As an example, for the schema {"properties": {"foo": {"title": "Foo", "description": "a list of strings", "type": "array", "items": {"type": "string"}}}, "required": ["foo"]}

the object {"foo": ["bar", "baz"]} is a well-formatted instance of the schema. The object {"properties": {"foo": ["bar", "baz"]}} is not well-formatted.


Here is the output schema:

```

{"properties": {"query": {"title": "Query", "type": "string"}, "is_malicious": {"title": "Is Malicious", "type": "boolean"}, "table_count": {"title": "Table Count", "type": "integer"}}, "required": ["query", "is_malicious", "table_count"]}

```
```

Using an Output Formatter

```
chain = chat_prompt | model | json_output_formatter
result = chain.invoke({"table": "users",
                       "columns": [...],
                       "format_instructions":
json_output_parser.get_format_instructions()
                       },
                      )
```

```
query='SELECT * FROM users WHERE id = 1' is_malicious=True table_count=1
```

The output is parsed into a useable object!

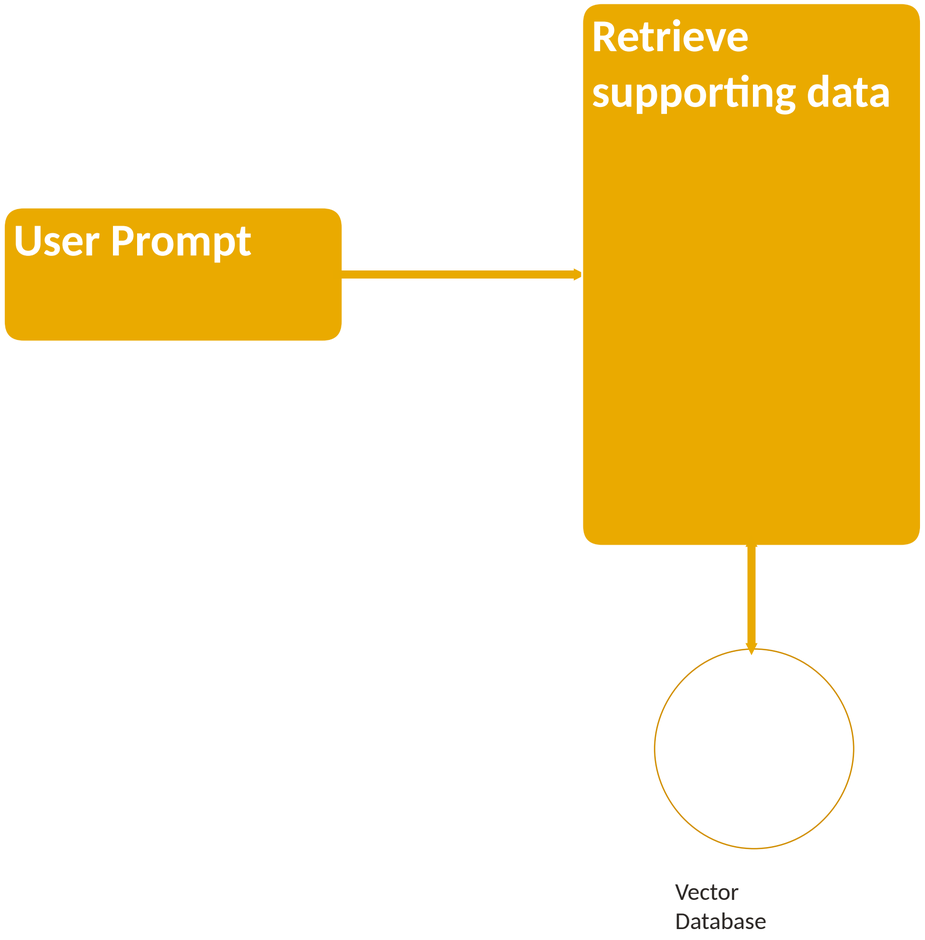
Implementing RAG with LangChain

RAG Workflow

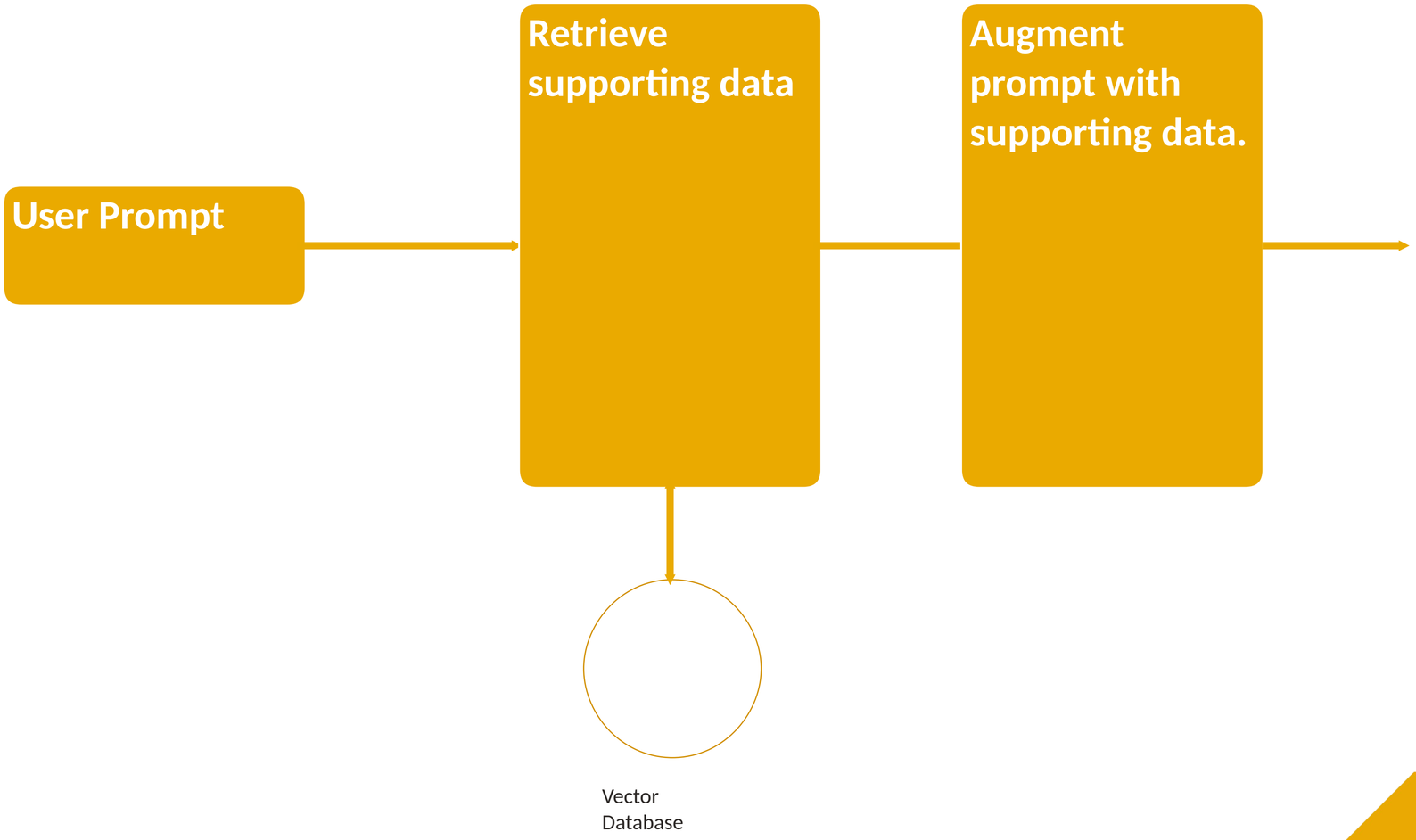
User Prompt

Vector
Database

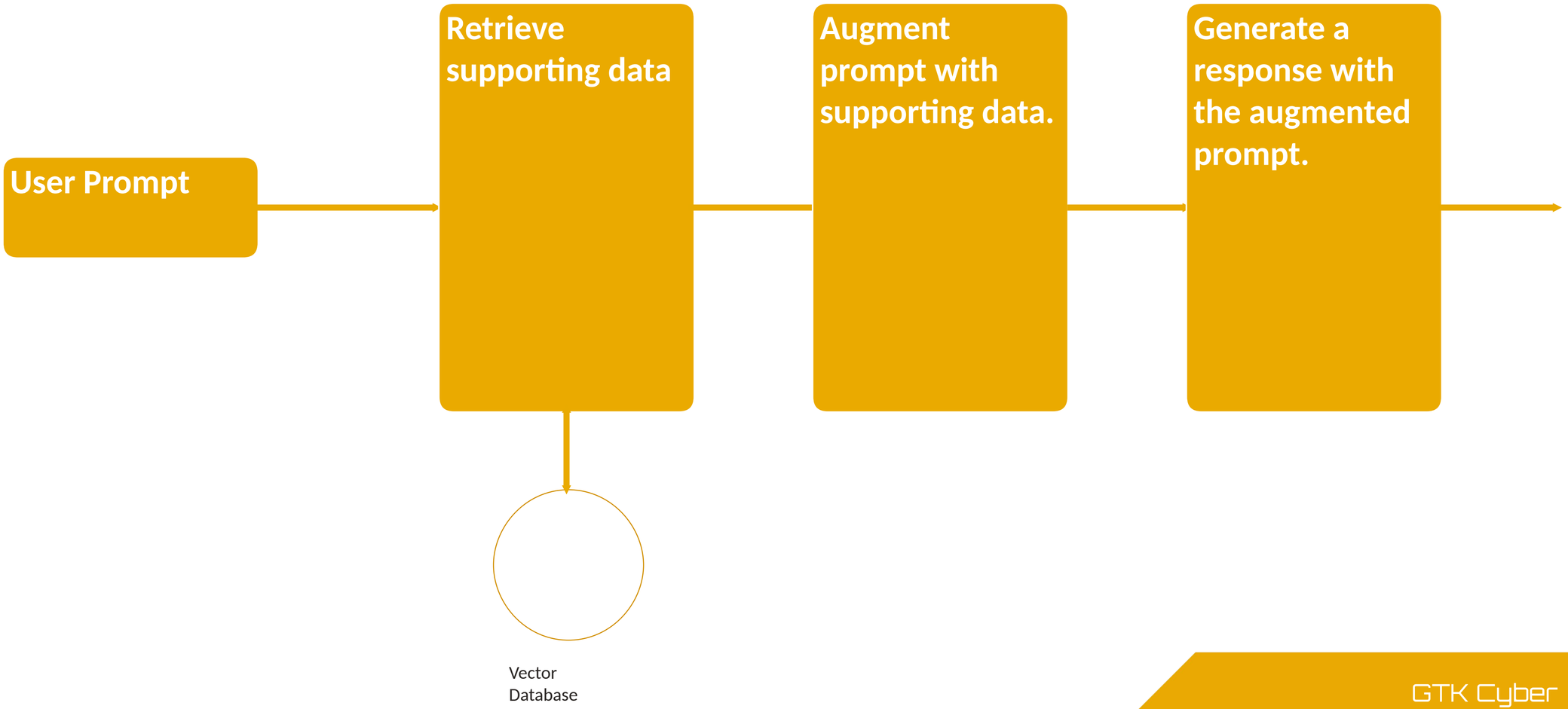
RAG Workflow



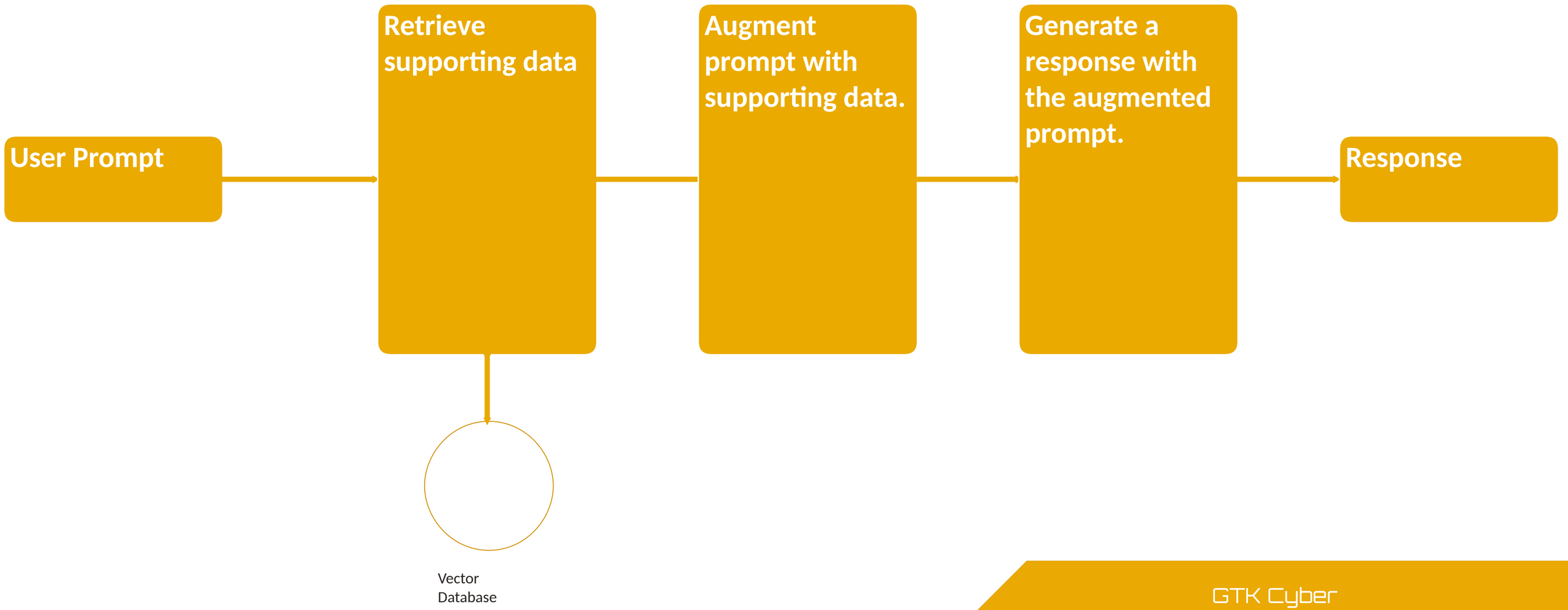
RAG Workflow



RAG Workflow



RAG Workflow



RAG Steps

1. Create "retriever" connection with document source.
2. Get user prompt
3. Send user prompt to vector database to retrieve context
4. Add context to prompt
5. Send augmented prompt to LLM
6. Get results

RAG: Creating the Retriever

```
# Connect to the Vector DB
db = Chroma.from_documents(docs, embeddings,
persist_directory="chroma_flex_store")

# Create retriever
retriever = db.as_retriever(search_type="similarity", search_kwargs={"k":4})
```

RAG: LLM & User Prompt

```
llm = ChatOpenAI(model="gpt-4o", temperature=0)

# Retrieve context from Chroma
context_docs = retriever.get_relevant_documents(user_request)
context_text = "\n\n".join([d.page_content for d in context_docs])
```

RAG: LLM & User Prompt

```
prompt = f"""
You are an AI assistant with access to the following context:
{context_text}

User request: {user_request}

Write a thoughtful, well-structured response that uses the context as needed,
but do not just copy it. If the context doesn't fully answer the request, provide
your best reasoning.
"""
```

RAG: LLM Response

```
user_request = input("\nEnter your request (or 'exit'): ")
   if user_request.lower() == "exit":
       break

prompt = rag_prompt(user_request)
response = llm.predict(prompt)
print("\nAI Response:\n", response)
```

Questions?

Lab

Please complete Worksheet 11.1: AI and RAG CVEs

AI Skills vs. Tools

Why This Distinction Matters

- Agents are only as good as what they can DO and what they KNOW how to do
- Tools give an agent access to systems: APIs, files, shells
- Skills give an agent expertise: procedures, judgment, house style
- Confusing the two produces bloated tool lists and thin, generic behavior
- Get it right and your agent behaves like a trained specialist

What Is a Tool?

- A single capability the model can invoke: a function with a name, inputs, and outputs
- Examples: read_file, run_sql_query, send_email, http_get
- Defined by a schema the model sees on every request
- The model decides when to call it and with which arguments
- A deterministic surface: the same call returns the same kind of result
- Every tool schema occupies context on every turn

What Is a Skill?

- A packaged unit of know-how the agent loads when a task calls for it
- Usually a folder: a SKILL.md plus optional scripts, templates, references
- Tells the agent how to do something well: the procedure, standards, gotchas
- Can bundle and orchestrate several tools toward an outcome
- Loaded on demand, so it stays cheap until a task needs it
- Think of a checklist and playbook a specialist wrote for the agent

Skills vs. Tools, Side by Side

- Purpose: a tool grants access; a skill grants expertise
- Form: a tool is a typed function; a skill is a folder of instructions
- Loading: a tool schema stays in context; a skill loads on demand
- Authoring: a tool needs code and schema; a skill needs clear prose
- Composition: a skill can call many tools; a tool holds no skill
- Analogy: a tool is the drill; a skill is knowing how to frame the wall

Anatomy of a Skill

- A folder named for the skill, containing a SKILL.md
- Frontmatter: a name and a description, where the description is the trigger
- Body: the instructions, covering when to use it, steps, rules, red flags
- Optional bundled files: scripts, templates, references, examples
- Progressive disclosure: the description stays visible, the body loads when triggered
- Keep the always-on part tiny and push detail into files opened as needed

Example Skill: 'commit-hygiene'

- commit-hygiene/
- SKILL.md
- references/conventional-commits.md

.

- --- SKILL.md ---
- name: commit-hygiene
- description: Use when writing a git commit message.
- Enforces the team format and splits unrelated changes.

.

- Steps:
- 1. Run git diff --staged; group changes by intent.
- 2. If two intents are staged, stop and suggest splitting.
- 3. Subject 50 chars max, imperative mood. Body explains the WHY.
- 4. See references/conventional-commits.md for type prefixes.

Cyber Example: 'triage-suspicious-url'

- triage-suspicious-url/
- SKILL.md
- references/ioc-format.md

.

- --- SKILL.md ---
- name: triage-suspicious-url
- description: Use when a link is reported by a user
- or flagged in an email or log.

.

- Steps:
- 1. Defang the URL before echoing it (hxxp, [.]).
- 2. Detonate in the sandbox tool; never open it live.
- 3. Pull whois age, TLS issuer, and hosting ASN.
- 4. Check the domain against the threat-intel lookups.
- 5. Classify benign, suspicious, or malicious, with reasons.
- 6. Emit IOCs per references/ioc-format.md and suggest a block.

Progressive Disclosure: the Reference File

- references/ioc-format.md (opened only at the report step)

.

- # IOC Output Format
- One object per indicator. Network values defanged.

.

- type: url | domain | ipv4 | sha256 | sender
- value: defanged string (hxxp, [.])
- verdict: benign | suspicious | malicious
- confidence: low | medium | high
- tlp: CLEAR | GREEN | AMBER | RED
- first_seen: ISO-8601 UTC
- source: email | proxy log | user report

.

- Rules:
- - No clickable links anywhere in the report.
- - SHA-256 preferred; add a block line when malicious.

How a Skill Gets Used

- 1. At rest: only the name and description sit in context
- 2. Match: the agent sees a task that fits the description
- 3. Load: the full SKILL.md body is read into context
- 4. Follow: the agent works the steps, opening reference files as needed
- 5. Act: the skill drives the underlying tools to finish the job
- Progressive disclosure means you pay for detail only when it applies

Where to Find Skills

- Anthropic Agent Skills docs and the anthropics/skills repo on GitHub
- Claude Code plugins and marketplaces, where skills ship inside plugins
- The Superpowers collection: brainstorming, debugging, TDD, and more
- Your own repo: a .claude/skills/ folder versioned with the project
- Team libraries that encode your house standards once for everyone
- Search GitHub for 'SKILL.md' to study how others structure real skills

Skill-Writing Tips

- Nail the description, since it is the trigger. Say exactly when to use it
- Write it for a competent new hire, with concrete, followable steps
- Keep SKILL.md lean and push tables and edge cases into reference files
- Prefer imperative checklists the agent can turn into todos
- Add red flags and negative examples to head off common failures
- Keep one skill to one job, and split it when it grows a second job
- Test it: give the agent a real task and watch the skill fire

Takeaways

- Tools give access and skills give expertise, so you want both
- A skill is packaged know-how, loaded on demand, in plain language
- Progressive disclosure keeps skills cheap until they become relevant
- The description is the trigger, so write it like you mean it

Using AI to Execute Code (Tools)

DANGER

- Calling functions from AI is inherently risky, especially if the AI accepts input from untrusted users.
- When implementing features that call functions, always ask yourself what could happen if this "goes rogue"?
- Remember that the AI can be somewhat unpredictable, so be sure to include deterministic safeguards.
- **This is especially true if your AI bot is connected to internal data sources.**

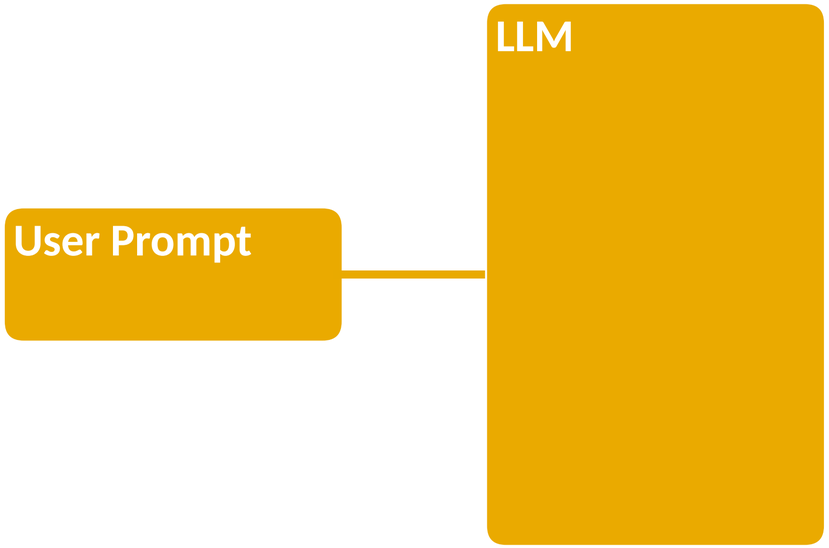


Function Calling Workflow



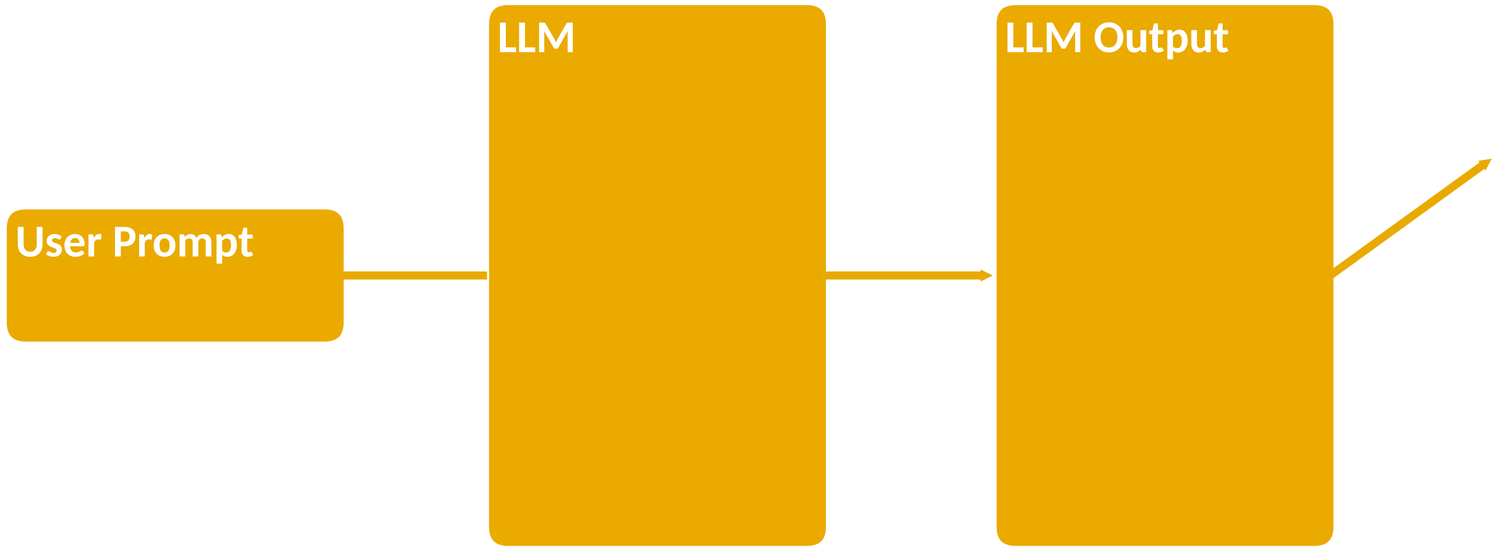
The tool will only be called if the model determines it needs to.

Function Calling Workflow



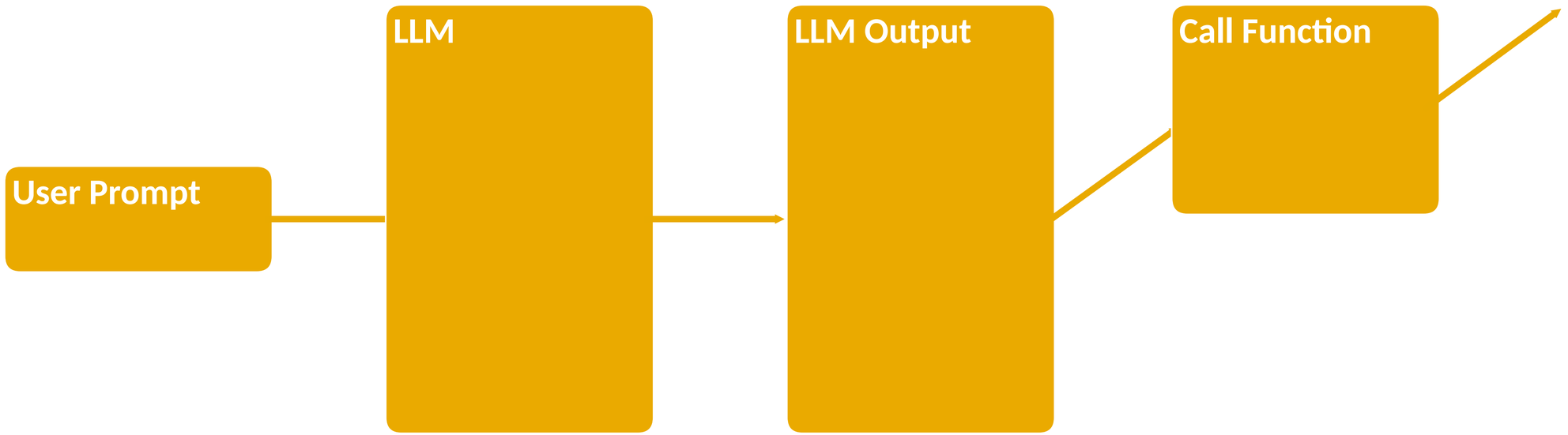
The tool will only be called if the model determines it needs to.

Function Calling Workflow



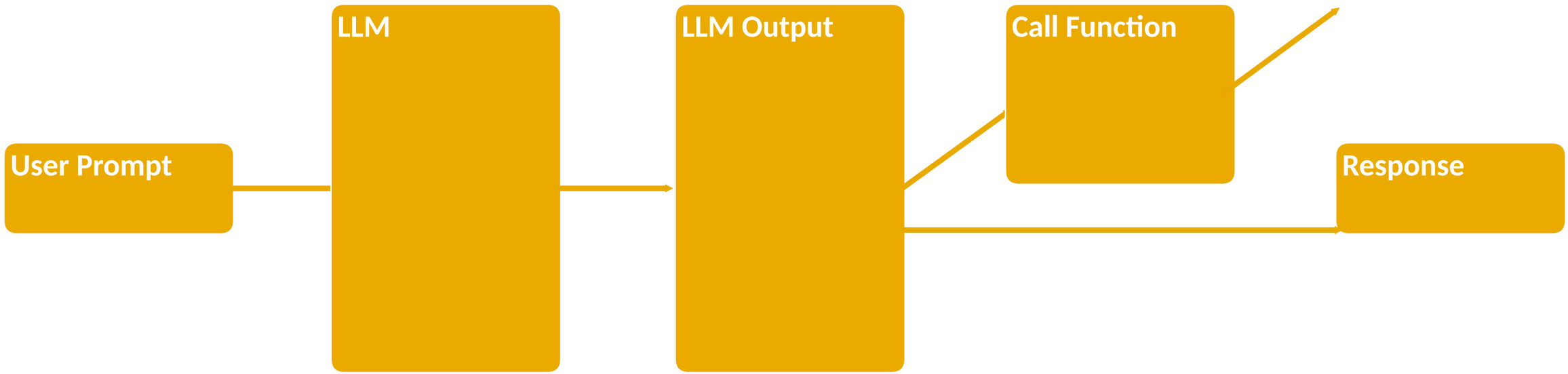
The tool will only be called if the model determines it needs to.

Function Calling Workflow



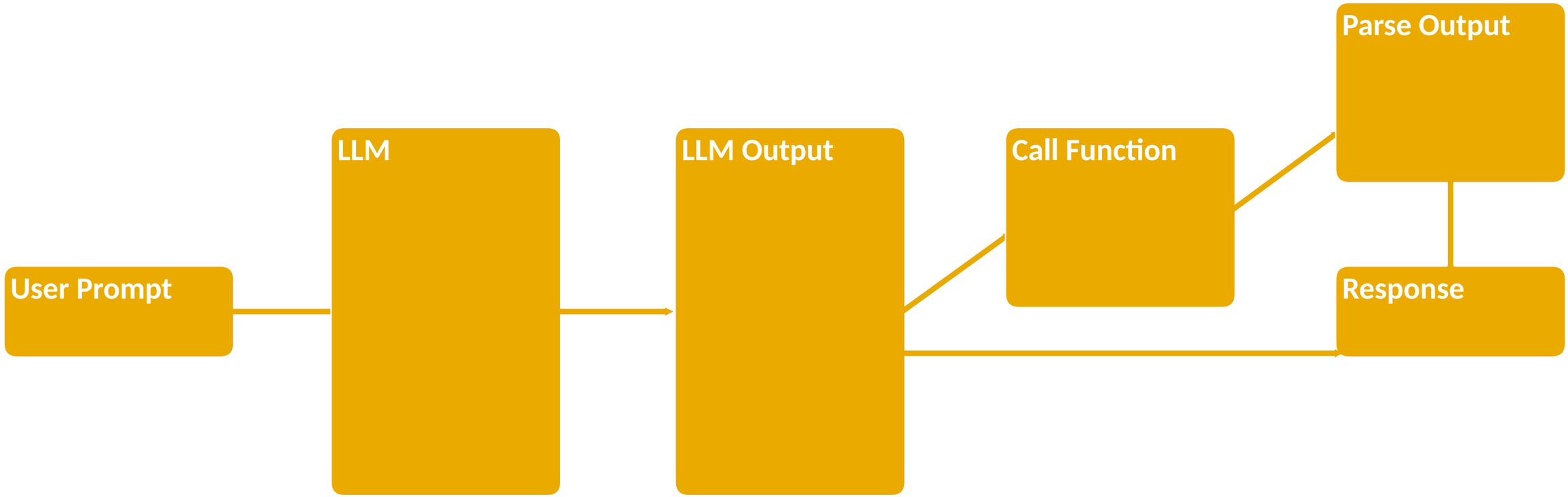
The tool will only be called if the model determines it needs to.

Function Calling Workflow



The tool will only be called if the model determines it needs to.

Function Calling Workflow



The tool will only be called if the model determines it needs to.

LangChain Function Calling

- With LangChain, you can call functions (tool) to get additional information or perform actions when appropriate.
- Not every LLM supports tool calling.
- LangChain has pre-built tools, but you can also define your own.
- A complete list of pre-built tools is available here:

https://python.langchain.com/docs/integrations/tools/

LangChain calls functions tools.

LangChain Tool Calling

- Examples include:
  - Search: (Google, Bing, DuckDuckGo, Others)
  - Interpreting code: (Azure container apps)
  - Productivity Tools: (Github, Gmail, O365, JIRA, Slack, More…)
  - Database Connectivity: (SQL Databases, Spark SQL, Cassandra SQL)
  - Others: (Bash shell, Google Finance, Google Drive, Pandas Dataframe, Salesforce,)

LangChain Pre-built Tool Example

```
# You'll need API tokens from GitHub
import getpass
import os

for env_var in [
    "GITHUB_APP_ID",
    "GITHUB_APP_PRIVATE_KEY",
    "GITHUB_REPOSITORY",
]:
    if not os.getenv(env_var):
        os.environ[env_var] = getpass.getpass()
```

LangChain Tool Example

```
from langchain_community.agent_toolkits.github.toolkit import GitHubToolkit
from langchain_community.utilities.github import GitHubAPIWrapper

# Instantiate the GitHub tool
github = GitHubAPIWrapper()
toolkit = GitHubToolkit.from_github_api_wrapper(github)
```

LangChain Tool Example

```
tools = toolkit.get_tools()

for tool in tools:
    print(tool.name)
```

- Get Issues
- Get Issue
- Comment on Issue
- List open pull requests (PRs)
- Get Pull Request
- Overview of files included in PR
- Create Pull Request
- List Pull Requests' Files
- Create File
- Read File
- Update File
- **Delete File**
- Overview of existing files in Main branch
- Overview of files in current working branch
- More...

LangChain Tool Example

```
# Create the chat model
llm = init_chat_model("gpt-4o-mini", model_provider="openai")

# Create a list of tools
tools = [tool for tool in toolkit.get_tools() if tool.name == "Get Issue"]
tools[0].name = "get_issue"

# Bind the tools to the agent
agent_executor = create_react_agent(llm, tools)
```

Discussion:

- You are a cybersecurity consultant and a client have an AI model which is connected to a database.
- Your task is to advise your client of the risks associated with their model.
- How would you recommend securing such a system?

Lab

Please complete Tool Usage Lab: https://ai.gtkcyber.com/tool-usage-lab

Implementing Tools

LangChain Custom Tool Example

- You can also create AI tools with your own custom functions.
- **Tool Creation**: Use the `@tool` decorator to create a `tool`. A tool is an association between a function and its schema.
- **Tool Binding**: The tool needs to be connected to a model that supports tool calling. This gives the model awareness of the tool and the associated input schema required by the tool.
- **Tool Calling**: When appropriate, the model can decide to call a tool and ensure its response conforms to the tool's input schema.
- **Tool Execution**: The tool can be executed using the arguments provided by the model.

https://python.langchain.com/docs/concepts/tool_calling/

LangChain Tool Creation

```
from langchain.tools import Tool

@tool
def my_tool(input: str) -> str:

    # Do something...


     return input[::-1]
```

LangChain Tool Binding

```
# Create the ChatGPT model
llm = ChatOpenAI(model="gpt-4", temperature=0, api_key=os.getenv("OPENAI_KEY"))

# Wrap your custom tool in LangChain's Tool class
my_tool = Tool(
   name="MyTool",
   func=my_tool,
   description="My really awesome tool.",
)

# Initialize the agent with the tool
agent = initialize_agent(
   tools=[phone_tool],
   llm=llm,
   agent=AgentType.ZERO_SHOT_REACT_DESCRIPTION,
   verbose=True
)
```

Tool Calling / Execution

```
response = agent.run(prompt)

# The model will decide whether to call the tool or not based on the user
input.
```

Questions?

Lab

Please complete Lab 11.2: Tool Usage